

CHRISCO DIGITAL ACADEMY

AI & TECHNOLOGY

The Practical Handbook Series

Python Programming

Learn to code from zero and build real, useful tools.

You will understand how programs think, write Python confidently, work with data and the internet, automate boring tasks, and finish by building a tool of your own.

8 CHAPTERS - SELF-STUDY GUIDE - PRACTICAL EXERCISES

Wambete Benjamin

Founder, CHRISCO Digital Academy - Jinja & Buikwe, Uganda - 2026 Edition

Python Programming

Learn to code from zero and build real, useful tools.

Published by CHRISCO Digital Academy under CHRISCO Youth Aflame, Jinja & Buikwe, Uganda. First edition, 2026. Written by Wambete Benjamin. This handbook is provided free to learners of the CHRISCO community hubs and may be shared for educational, non-commercial use. Contact: shambetz@gmail.com - WhatsApp +256 763 393 021 / +256 755 467 513.

A note before you begin

Coding is one of the highest-paid, most portable skills on earth, and Python is the friendliest doorway into it. If you can follow a recipe and think step by step, you can learn to program. No maths genius required.

This book is hands-on. Reading about code is like reading about swimming - you must get in the water. Type every example yourself, break it, fix it. The errors are not failures; they are the lessons.

By the end you will not just 'know some Python' - you will have built something that works. That project is your proof, and proof is what gets you hired.

Contents

- 01 Introduction to programming
 - 02 Python syntax and variables
 - 03 Conditions and loops
 - 04 Functions and modules
 - 05 Working with files and data
 - 06 Introduction to APIs
 - 07 Building automation scripts
 - 08 Build your Python tool
-

HOW TO USE THIS BOOK

Read one chapter at a time. Don't just read - do the 'Try This' exercise in each chapter before you move on, and keep your answers in a notebook. The 'In Practice' stories show how the idea works in real life. By the last page you should have real work to show, not just notes.

Introduction to programming

A program is just a set of clear instructions a computer follows exactly, in order. Computers are fast but literal - they do precisely what you say, not what you meant. Learning to code is really learning to think clearly: to break a problem into small, unambiguous steps. That thinking skill is valuable far beyond programming.

Python is popular because it reads almost like English and is used everywhere - websites, data, AI, automation. You will need a place to write and run code; you can start free in your browser (such as Google Colab or an online Python editor) with no installation, which means you can learn even on a modest device.

IN PRACTICE

Think of a recipe: 'boil water, add rice, wait 15 minutes.' Miss a step or change the order and dinner fails. Code is the same - order and precision are everything. Good programmers are just very careful step-thinkers.

TRY THIS

Open a free online Python editor. Write and run your very first line: `print("Hello, my name is ...")`. Change the text and run it again.

Key takeaways

- A program is precise, ordered instructions.
- Coding trains clear step-by-step thinking.
- Start free in the browser - no fancy device needed.

Python syntax and variables

Syntax is the grammar of code - the rules for how you write it. Python is strict about some things (spelling, indentation) and forgiving about others. A variable is a labelled box that stores a value: `name = "Sarah"` or `age = 20`. You put data in variables so your program can use and change it later.

Python has a few basic data types you will use constantly: text (strings), whole numbers (integers), decimals (floats), and true/false (booleans). Understanding these, and how to create and print variables, is the foundation everything else is built on. Go slowly here; a solid base makes the rest easy.

IN PRACTICE

`price = 5000` and `quantity = 3`, then `total = price * quantity` gives 15000. You just did shop maths in code - and the computer will redo it instantly for any numbers you give it.

TRY THIS

Create variables for your name, age and town. Print a sentence using them, like: `print(name + " is " + str(age) + " years old")`.

Key takeaways

- Variables store data in named boxes.
- Know strings, integers, floats and booleans.
- A solid base makes everything after it easier.

Conditions and loops

Conditions let your program make decisions using `if / else`: 'if the balance is enough, allow the purchase, else refuse it.' This is how software responds differently to different situations. You compare values (equal to, greater than, less than) and act on the result. Decisions are what make a program feel intelligent rather than fixed.

Loops let your program repeat work without you writing it a hundred times. A `for`-loop runs through a list of items; a `while`-loop repeats until a condition changes. Together, conditions and loops are the engine of almost every program - deciding and repeating are most of what code does.

IN PRACTICE

`for student in class_list: print(student)` prints every name in a list, whether it has five names or five thousand. Loops turn a tiny bit of code into unlimited work done instantly.

TRY THIS

Write a loop that prints numbers 1 to 10, and add a condition that prints 'even' next to the even ones.

Key takeaways

- `if/else` lets programs make decisions.
- Loops repeat work automatically.
- Deciding and repeating are most of coding.

Functions and modules

A function is a reusable block of code you name once and use many times. Instead of repeating the same steps, you wrap them in a function and just call it: `greet()` or `calculate_total(price, quantity)`. Functions keep your code short, organised and easy to fix - change it in one place, and it updates everywhere it is used.

Modules are collections of ready-made functions other people wrote that you can import and use for free. Need random numbers, dates, or maths? There is a module for it. This is a superpower: you rarely build from scratch, you stand on the work of a huge global community. Learning to find and use modules multiplies what you can do.

IN PRACTICE

Instead of writing the same tax calculation in ten places, you write one function `calculate_tax(amount)` and call it wherever needed. Fix the formula once and every calculation across your program is corrected.

TRY THIS

Write a function that takes a price and returns the price plus 18% tax. Call it with three different prices.

Key takeaways

- Functions are reusable, named blocks of code.
- They keep code short and easy to fix.
- Modules let you reuse the world's code for free.

Working with files and data

Real programs handle data - reading it in, processing it, and saving results. You will learn to open and read files, write to them, and work with lists and dictionaries (which store data as key-value pairs, like a contact's name mapped to their phone number). Handling data is where code stops being toy examples and starts being useful.

A common, valuable skill is working with spreadsheet-style data (CSV files): reading rows, doing calculations, and writing a report. Being the person who can turn a messy pile of data into a clean summary in seconds is genuinely marketable - many jobs need exactly that, and few people can do it.

IN PRACTICE

A small business had a year of sales in a spreadsheet and no time to analyse it. A short Python script read the file, added up sales per month, and printed a summary in seconds - work that would take hours by hand.

TRY THIS

Create a list of five products with prices (use a dictionary). Loop through it and print each product with its price, then print the total.

Key takeaways

- Programs read, process and save data.
- Lists and dictionaries organise information.
- Turning messy data into summaries is marketable.

Introduction to APIs

An API is a way for your program to talk to another program over the internet and get data back - live weather, exchange rates, football scores, or the ability to send a message. Instead of building everything yourself, you request data from services that already have it. This opens a huge world of possibilities with little code.

You will learn to send a request to an API and read the response (usually in a format called JSON, which is just organised text). Once you can consume an API, your programs can pull in real, live information from anywhere - the difference between a program that only knows what you typed and one connected to the whole internet.

IN PRACTICE

With a few lines calling a free currency API, a program can fetch today's USD-to-UGX rate and instantly convert prices - always up to date, never hand-typed. That small skill powers countless real apps.

TRY THIS

Find a simple free public API (like a random-joke or currency API). Write code to request it and print one piece of the data it returns.

Key takeaways

- APIs let your code fetch data from other services.
- Responses often come as JSON (organised text).
- APIs connect your program to the whole internet.

Building automation scripts

The most immediately useful thing you can do with Python is automate boring, repetitive tasks: renaming hundreds of files, cleaning a spreadsheet, sending routine emails, or combining reports. A script that saves an hour a day, every day, is worth more than it looks - and it is exactly the kind of quiet skill employers pay for.

Automation is also how you build a portfolio fast. Solve a real annoyance in your own life or a friend's small business with a script, and you have both a useful tool and proof of skill. You do not need a grand app - a small script that reliably does a real job is genuinely impressive.

IN PRACTICE

A student wrote a 20-line script that renamed and organised hundreds of photos by date. A shopkeeper saw it, asked for a stock-total script, and paid for it. The first client came from solving one small, real problem.

TRY THIS

Think of one repetitive task you or someone you know does by hand. Sketch the steps, then write a simple script that does part of it automatically.

Key takeaways

- Automating boring tasks is Python's quickest payoff.
- Small time-saving scripts have real value.
- Solve real annoyances to build a portfolio.

Build your Python tool

Your final project is a working tool you build yourself - a budget calculator, a data summariser, a small automation, a simple quiz game, anything that solves a real need. Choose something slightly beyond your comfort zone but finishable. Building end-to-end teaches you more than any tutorial, because you must connect every piece yourself.

Expect to get stuck - that is normal and where real learning happens. Break the problem into tiny steps, test each piece, and search for errors (every programmer relies on searching and now on AI assistants). When it finally runs, you will have crossed the line from 'learning to code' to 'I can build things' - and you will have proof to show.

IN PRACTICE

One learner built a simple loan-interest calculator for boda riders saving as a group. It was small, but it worked, solved a real need, and became the project that got her a junior data role. Finished beats fancy.

TRY THIS

Plan and build one small Python tool this week. Break it into steps, build one step at a time, and get the whole thing working end to end.

Key takeaways

- Building end-to-end teaches the most.
- Getting stuck is where you learn.
- A finished small tool is real, hireable proof.



KEEP GOING

Skills pay for life. Now go use them.

Finishing the reading is not the goal - finishing the project is. Pick the one exercise in this book that scared you most and do it this week. Then bring your work to your CHRISCO facilitator or post it online. That single habit - shipping real work - is what separates the youth who earn from the youth who only dream.

CHRISCO Digital Academy

Jinja & Buikwe, Uganda - shambetz@gmail.com

WhatsApp +256 763 393 021 / +256 755 467 513 - Under CHRISCO Youth Aflame - Founder:
Wambete Benjamin